LAPORAN PRAKTIKUM

STRUKTUR DATA

"IMPLEMENTASI ADT ARRAYLIST"

disusun oleh

Hafizh Habibullah

2511531002

Dosen Pengampu:

Dr. Wahyudi, S.T., M.T.

Asisten Praktikum:

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kita panjatkan ke hadirat Allah SWT. karena atas rahmat dan karunia-Nya laporan Praktikum Struktur Data ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas praktikum pada mata kuliah Struktur Data dengan topik Implementasi ADT *ArrayList*. Melalui laporan ini, penulis berusaha menjelaskan hasil praktikum yang meliputi penjelasan mengenai kode program, tujuan, serta cara kerja program.

Penulis sangat mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di kemudian hari. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, 15 April 2026

Hafizh Habibullah

DAFTAR ISI

KATA PENGANTAR 1
DAFTAR ISI 2
BAB I PENDAHULUAN 3
1.1 Latar Belakang 3
1.2 Tujuan 3
1.3 Manfaat 3
BAB 2 PEMBAHASAN 4
2.1 Struktur Dasar ArrayList 4
2.2 Kelas Mahasiswa 5
2.3 Kelas MahasiswaDriver 6
2.4 Kelas DaftarKata 9
2.5 Kelas DaftarKataDriver 11
BAB III KESIMPULAN 13
3.1 Kesimpulan 13
DAFTAR PUSTAKA 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan *software*, pengelolaan data secara efisien merupakan hal yang sangat penting. Salah satu cara mewujudkan hal tersebut adalah menggunakan Struktur Data yang tepat. *Abstract Data Type* (ADT) merupakan model matematika dari suatu tipe data yang mendefinisikan perilaku dari struktur data tersebut dari sudut pandang pengguna tanpa memperhatikan detail implementasinya.

ArrayList merupakan salah satu implementasi dari ADT List yang paling banyak digunakan. Berbeda dengan *Array* Statis yang ukurannya tetap, *ArrayList* mampu mengubah ukurannya secara dinamis sesuai kebutuhan. Hal ini menjadikan *ArrayList* sebagai struktur data yang fleksibel dan efisien untuk menyimpan kumpulan elemen yang jumlahnya tidak dapat ditentukan dari awal.

1.2 Tujuan

Adapun tujuan dari praktikum ini adalah:

1. Memahami konsep dasar Abstract Data Type (ADT) dan kaitannya dengan struktur data ArrayList.
2. Mampu mengidentifikasi dan menjelaskan operasi-operasi dasar pada ADT ArrayList.
3. Mampu mengimplementasikan ADT ArrayList menggunakan bahasa pemrograman Java.

1.3 Manfaat

Manfaat dari Praktikum ini adalah:

1. Menerapkan konsep ADT ArrayList dalam menyelesaikan permasalahan pemrograman yang melibatkan pengelolaan data dinamis.
2. Mendapatkan pemahaman yang lebih baik mengenai abtraksi data sebagai dasar perancangan perangkat lunak yang terstruktur.
3. Memiliki keterampilan dalam mengimplementasikan struktur data ArrayList secara mandiri, tidak hanya bergantung pada library bawaan.

BAB 2

PEMBAHASAN

2.1 Struktur Dasar ArrayList

```java
package pekan2_2511531002;

import java.util.ArrayList;

public class ArrayList1_2511531002 {
    public static void main(String[] args) {
        // Size of the ArrayList
        int n = 5;
        // Declaring the ArrayList with initial size n
        ArrayList<Integer> arrli = new ArrayList<Integer>(n);
        // Appending  new elements at the end of the list
        for (int i = 1; i ≤ n; i++)
            arrli.add(i);
        // Printing elements
        System.out.println(arrli);
        // Remove element at index 3
        arrli.remove(3);
        // Displaying the ArrayList after deletion
        System.out.println(arrli);
        // Printing elements one by one
        for (int i = 0; i < arrli.size(); i++)
            System.out.print(arrli.get(i) + " ");
    }
}
```

Kode Program 2. 1 - Kode Program ArrayList_2511531002

Kode program ini dimulai dengan mendeklarasikan sebuah variabel n dengan nilai 5. Ini akan menjadi ukuran dari *ArrayList* yang akan dibuat. Kemudian, dibuat sebuah objek *ArrayList* dengan nama **arrli,** dengan tipe data Integer.

Blok program berikutnya dimulai dengan for-loop sederhana dari i-n. Disini akan dimasukkan angka 1-5 ke dalam **arrli**. Baris berikutnya akan mencetak nilai yang terdapat pada **arrli**. Kemudian, dihapuslah sebuah elemn dengan indeks ke-3. Lalu, dicetakkan kembali **arrli** dengan nilai yang sudah dihapus.

Blok program terakhir juga merupakan sebuah for-loop sederhana dari 0 hingga banyak elemen dari **arrli** saat ini. Karena banyak elemen dari **arrli** adalah 4 elemen, maka for-loop tersebut akan beriterasi sebanyak 4 kali. Untuk for-loop ini, ia akan mengambil semua elemen dari **arrli** menggunakan method **get()**, dan dipisahkan dengan *whitespace* **(“ “).**

Hasil dari program tersebut adalah sebagai berikut:

```
Console   Progress
<terminated> ArrayList1_25115310
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

Gambar 2. 1 – Output dari Kode Program ArrayList_2511531002

2.2 Kelas Mahasiswa

```java
package pekan2_2511531002;

public class Mahasiswa_2511531002 {
    String nim;
    String nama;
    String prodi;
    Mahasiswa_2511531002(String nim, String nama, String prodi) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
    }
    @Override
    public String toString() {
        return "NIM: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;
    }
}
```

Kode Program 2. 2 – Kode Program Mahasiswa_2511531002

Kelas Mahasiswa merupakan class model yang merepresentasikan data seorang mahasiswa dengan tiga atribut, yaitu nim, nama, dan prodi, yang dimana ketiganya bertipe String. Class ini memiliki sebuah konstruktor yang menerima ketiga atribut tersebut sebagai parameter dan menginisialisasinya menggunakan keyword “this” untuk membedakan atribut class dengan parameter yang masuk.

Method toString() di-*override* dari class Object bawaan Java agar ketika objek Mahasiswa dicetak, outputnya berupa string yang terformat rapi menampilkan NIM, nama, dan prodi mahasiswa tersebut.

2.3 Kelas MahasiswaDriver

```java
package pekan2_2511531002;

import java.util.ArrayList;
import java.util.Scanner;
public class MahasiswaDriver_2511531002 {
    // 1. Method untuk menampilkan menu
    public static void tampilkanMenu_2511531002() {
        System.out.println("\nMenu:");
        System.out.println("1. Tambah Mahasiswa");
        System.out.println("2. Tampilkan Semua Mahasiswa");
        System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
        System.out.println("5. Keluar");
    }
```

Kode Program 2. 3 – Method untuk Menampilkan Menu

Method ini hanya bertugas untuk mencetak pilihan menu ke layar, mulai dari opsi tambah, tampilkan, hapus, cari, hingga keluar dari menu.

```java
// 2. Method untuk tambah mahasiswa
public static void tambahMahasiswa_2511531002(ArrayList<Mahasiswa_2511531002> list, Scanner sc) {
    System.out.print("Masukkan NIM: ");
    String nim = sc.nextLine();
    System.out.print("Masukkan Nama: ");
    String nama = sc.nextLine();
    System.out.print("Masukkan Prodi: ");
    String prodi = sc.nextLine();
    list.add(new Mahasiswa_2511531002(nim, nama, prodi));
    System.out.println("Mahasiswa berhasil ditambahkan.");
}
```

Kode Program 2. 4 – Method untuk Menerima Input

Method ini menerima dua parameter yaitu list dan sc. Program meminta Input NIM, Nama, dan Prodi dari pengguna, lalu membuat objek Mahasiswa baru dari ketiga input tersebut dan langsung menambahkannya ke dalam list menggunakan add().

```java
// 3. Method untuk tampilkan semua data
public static void tampilkanSemuaMahasiswa_2511531002(ArrayList<Mahasiswa_2511531002> list) {
    if (list.isEmpty()) {
        System.out.println("Daftar mahasiswa kosong.");
    } else {
        System.out.println("Data Mahasiswa:");
        for (Mahasiswa_2511531002 mhs : list) {
            System.out.println(mhs);
        }
    }
}
```

Kode Program 2. 5 – Method untuk Menampilkan Semua Data pada Daftar

Sebelum mencetak, method ini memeriksa terlebih dahulu apakah list kosong menggunakan isEmpty(). Jika kosong, ditampilkan pesan bahwa daftar kosong. Jika tidak, dilakukan iterasi menggunakan *for-each loop* dan setiap objek akan dicetak. For-each loop adalah versi sederhana dari for biasa yang dirancang khusus untuk iterasi koleksi atau array tanpa perlu mengurus indeks secara manual.

```java
// 4 . Method untuk hapus mahasiswa berdasarkan NIM
public static void hapusMahasiswa_2511531002(ArrayList<Mahasiswa_2511531002> list, Scanner sc) {
    System.out.print("Masukkan NIM yang akan dihapus: ");
    String nimHapus = sc.nextLine();
    boolean removed = list.removeIf(mhs -> mhs.nim.equals(nimHapus));

    if (removed) {
        System.out.println("Data dengan NIM " + nimHapus + " berhasil dihapus.");
    } else {
        System.out.println("NIM tidak ditemukan.");
    }
}
```

Kode Program 2. 6 – Method untuk Menghapus Mahasiswa berdasarkan NIM

mhs → mhs.nim.equals(nimHapus) merupakan contoh ekspresi lambda pada Java. Ekspresi ini mempersingkat penulisan for-loop menjadi satu baris program saja. Versi for-loop normalnya adalah sebagai berikut:

```java
for (int i = 0; i < list.size(); i++ ) {
      if (list.get(i).nim.equals(nimHapus)) {
            list.remove(i);
            break;
      }
}
```

Method ini menggunakan removeIf() dengan lambda expression mhs → mhs.nim.equals(nimHapus) untuk menghapus elemen yang NIM-nya cocok. removeIf() mengembalikan nilai boolean, true jika ada elemen yang berhasil dihapus, false jika tidak ditemukan, sehingga bisa langsung dipakai untuk menampilkan pesan hasil operasi.

```java
// 5. Method untuk cari mahasiswa berdasarkan NIM
public static void cariMahasiswa_2511531002(ArrayList<Mahasiswa_2511531002> list, Scanner sc) {
    System.out.print("Masukkan NIM yang dicari: ");
    String nimCari = sc.nextLine();
    boolean ditemukan = false;

    for (Mahasiswa_2511531002 mhs : list ) {
        if (mhs.nim.equals(nimCari)) {
            System.out.println("Hasil pencarian: " + mhs);
            ditemukan = true;
            break;
        }
    }

    if (!ditemukan) {
        System.out.println("NIM tidak ada.");
    }
}
```

Kode Program 2. 7 – Method untuk Mencari Mahasiswa Berdasarkan NIM

Method ini melakukan pencarian dengan iterasi satu per satu menggunakan *for each loop*, membandingkan NIM tiap objek dengan NIM yang dicari menggunakan equals(). Jika ditemukan, data langsung dicetak dan perulangan dihentikan dengan break. Variabel "ditemukan" digunakan sebagai flag untuk mendeteksi kasus NIM tidak ada di list.

```java
    public static void main(String[] args) {
        ArrayList<Mahasiswa_2511531002> mahasiswaList = new ArrayList<>();
        Scanner scanner = new Scanner(System.in);
        int choice;

        do {
            tampilkanMenu_2511531002();
            System.out.print("Pilih menu: ");
            choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
            case 1:
                tambahMahasiswa_2511531002(mahasiswaList, scanner);
                break;
            case 2:
                tampilkanSemuaMahasiswa_2511531002(mahasiswaList);
                break;
            case 3:
                hapusMahasiswa_2511531002(mahasiswaList, scanner);
                break;
            case 4:
                cariMahasiswa_2511531002(mahasiswaList, scanner);
                break;
            case 5:
                System.out.println("Keluar dari program.");
                break;
            default:
                System.out.println("Pilihan tidak valid.");
            }
        } while (choice ≠ 5);
        scanner.close();
    }
}
```

Kode Program 2. 8 – Method Utama

Program berjalan dalam struktur do-while yang terus mengulang selama input yang diberikan tidak sama dengan 5. Setiap iterasi menampilkan menu, membaca pilihan dengan nextInt(), lalu scanner.nextLine() dipanggil setelahnya untuk membersihkan sisa newline di buffer agar input String berikutnya tidak terganggu. Pilihan kemudian diteruskan ke switch yang memanggil method yang sesuai. Saat pengguna memilih opsi 5, loop terhenti dan scanner.close() dipanggil untuk menutup resource.

2.4 Kelas DaftarKata

```
4  public class DaftarKata_2511531002 {
5      private final ArrayList<String> data;
6      // Konstruktor: inisialisasi list kosong
7      public DaftarKata_2511531002() {
8          this.data = new ArrayList<>();
9      }
```

Kode Program 2. 9 – Inisialisasi ArrayList

Class ini memiliki satu atribut, yaitu “data”, bertipe ArrayList<String> yang dideklarasikan private final. Private berarti atribut ini tidak bisa diakses langsung dari luar class, sedangkan final berarti referensi data tidak bisa diganti ke objek ArrayList lain setelah diinisialisasi. Konstruktor menginisialisasi data sebagai list kosong.

```
10     /** Menambahkan elemen di akhir list. */
11     public void tambah_2511531002(String elemen) {
12         data.add(elemen);
13     }
14     /** Menambahkan elemen pada indeks tertentu (menyisipkan). */
15     public void tambahPada_2511531002(int index, String elemen) {
16         data.add(index, elemen);
17     }
```

Kode Program 2. 10 – Menamhahkan Elemen ke List

Keduanya wrapper dari method add() milik ArrayList. Wrapper adalah method yang “membungkus” method lain tanpa menambah logika. Tujuannya hanya untuk menyembunyikan detail implementasi dari pengguna class. tambah() menambahkan elemen di akhir list, sedangkan tambahPada() menerima parameter

indeks tambahan untuk menyisipkan elemen di posisi tertentu. Elemen yang sudah ada di indeks tersebut dan seterusnya otomatis bergeser ke kanan.

```
public void ubahElemen_2511531002(int index, String nilaiBaru) {
    data.set(index, nilaiBaru);
}
```

Kode Program 2. 11 – Mengubah Elemen pada Indeks Tertentu

Menggunakan set(index, nilaiBaru) untuk mengganti elemen pada indeks tertentu dengan nilai baru. Berbeda dengan add() yang menyisipkan, set() menimpa elemen yang sudah ada tanpa menggeser elemen lain.

```
public String hapusElemen_2511531002(int index) {
    return data.remove(index);
}
```

Kode Program 2. 12 – Menghapus elemen pada Indeks tertentu

Memanggil remove(index) dan mengembalikan elemen yang dihapus sebagai String. Selain menghapus, method ini juga memberi tahu pemanggil elemen mana yang baru saja dihilangkan dari list.

```
public void iterasiCetak_2511531002() {
    for (int i = 0; i < data.size(); i++) {
        System.out.print(data.get(i)+" ");
    }
}
```

Kode Program 2. 13 – Menampilkan semua elemen

Mencetak semua elemen list menggunakan for biasa dengan indeks. Menggunakan data.size() sebagai batas agar iterasi selalu menyesuaikan ukuran list yang bersifat dinamis.

```java
    /** Mengambil elemen berdasarkan indeks. */
    public String get(int index) {
        return data.get(index);
    }

    // Representasi string agar mudah dicetak
    @Override
    public String toString() {
        return data.toString();
    }
}
```

Kode Program 2. 14 – Mengambil Elemen pada Indeks tertentu; Mengubah ArrayList menjadi tipe data String

get() adalah getter sederhana untuk mengakses elemen di indeks tertentu dari luar class. toString() di-*override* untuk mendelegasikan representasi String-nya langsung ke data.toString() milik ArrayList.

2.5 Kelas DaftarKataDriver

```java
package pekan2_2511531002;

public class DaftarKataDriver_2511531002 {
    public static void main(String[] args) {
        DaftarKata_2511531002 a1 = new DaftarKata_2511531002();

        // Menambah elemen (akhir)
        a1.tambah_2511531002("Kami");
        a1.tambah_2511531002("Informatika");

        // Menyisipkan elemen pada indeks 1
        a1.tambahPada_2511531002(1,"Mahasiswa");

        // Cetak isi awal
        System.out.println("Awal          : " + a1);

        // Mengubah elemen (index 1)
        a1.ubahElemen_2511531002(1, "Departemen");
        System.out.println("Setelah Ubah  : " + a1);

        // Menghapus elemen (hapus index 0)
        String terhapus = a1.hapusElemen_2511531002(0);
        System.out.println("Terhapus      : " + terhapus);
        System.out.println("Setelah Hapus : " + a1);

        // Iterasi pada ArrayList (cetak setiap elemen)
        System.out.print("Iterasi       : ");
        a1.iterasiCetak_2511531002();
        System.out.println();
    }
}
```

Kode Program 2. 15 – Program Driver untuk Mengetes Kelas DaftarKata

Pertama-tama, dua elemen ditambahkan di akhir list, yaitu “Kami” dan “Informatika”, sehingga list menjadi [Kami, Informatika]. Lalu, “Mahasiswa” disisipkan di indeks 1, sehingga list berubah menjadi [Kami, Mahasiswa, Informatika] dan dicetak sebagai kondisi awal.

Selanjutnya, elemen di indeks 1 diubah dari “Mahasiswa” menjadi “Departemen” menggunakan ubahElemen(), lalu elemen di indeks 0, yaitu “Kami”, dihapus menggunakan hapusElemen(). Nilai yang terhapus ditangkap ke variabel terhapus dan dicetak tersendiri, sehingga list akhir menjadi [Departemen, Informatika].

Terakhir, isi list dicetak ulang menggunakan iterasiCetak() yang mencetak tiap elemen satu per satu secara horizontal.

BAB III
KESIMPULAN

3.1 Kesimpulan

ArrayList merupakan implementasi dari ADT List yang bersifat dinamis. Artinya, ukurannya dapat bertambah atau berkurang secara otomatis sesuai kebutuhan, berbeda dengan Array Statis yang ukurannya harus ditentukan dari awal. ArrayList menyediakan berbagai operasi dasar seperti penambahan elemen di akhir list maupun pada indeks tertentu (add()), pengubahan elemen (set()), penghapusan elemen (remove()), dan pengaksesan elemen (get()), yang semuanya dapat digunakan secara langsung maupun dibungkus dalam method custom melalui prinsip enkapulasi.

Konsep enkapulasi diterapkan dengan menjadikan atribut ArrayList sebagai private sehingga akses dan manipulasi data hanya dapat dilakukan melalui method yang telah disediakan, menjadi kode lebih terstruktur dan mudah dipelihara. Selain itu, ArrayList dapat menyimpan objek dari class buatan sendiri seperti Mahasiswa, sehingga sangat fleksibel untuk merepresentasikan data yang lebih kompleks dalam sebuah program. Secara keseluruhan, pemahaman terhadap ADT ArrayList memberikan fondasi yang kuat dalam merancang struktur data yang efisien dan berorientasi objek.

DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data.